

1) Primer protocolo de aplicación y su transporte

El primero es DNS (la PC resuelve `www.mipc.com` → IP antes de navegar). Su transporte es UDP (puerto 53), como se ve en la Layer 4 del paquete.

PDU Information at Device: PC

OSI Model

Outbound PDU Details

At Device: PC
Source: PC
Destination: 192.168.0.3

In Layers

Layer7

Layer6

Layer5

Layer4

Layer3

Layer2

Layer1

Out Layers

Layer 7: DNS

Layer6

Layer5

Layer 4: UDP Src Port: 1025, Dst Port: 53

Layer 3: IP Header Src. IP: 192.168.0.1, Dst. IP: 192.168.0.3

Layer 2:

Layer1

1. The device encapsulates the PDU into an UDP segment.

Challenge Me

<< Previous Layer

Next Layer >>

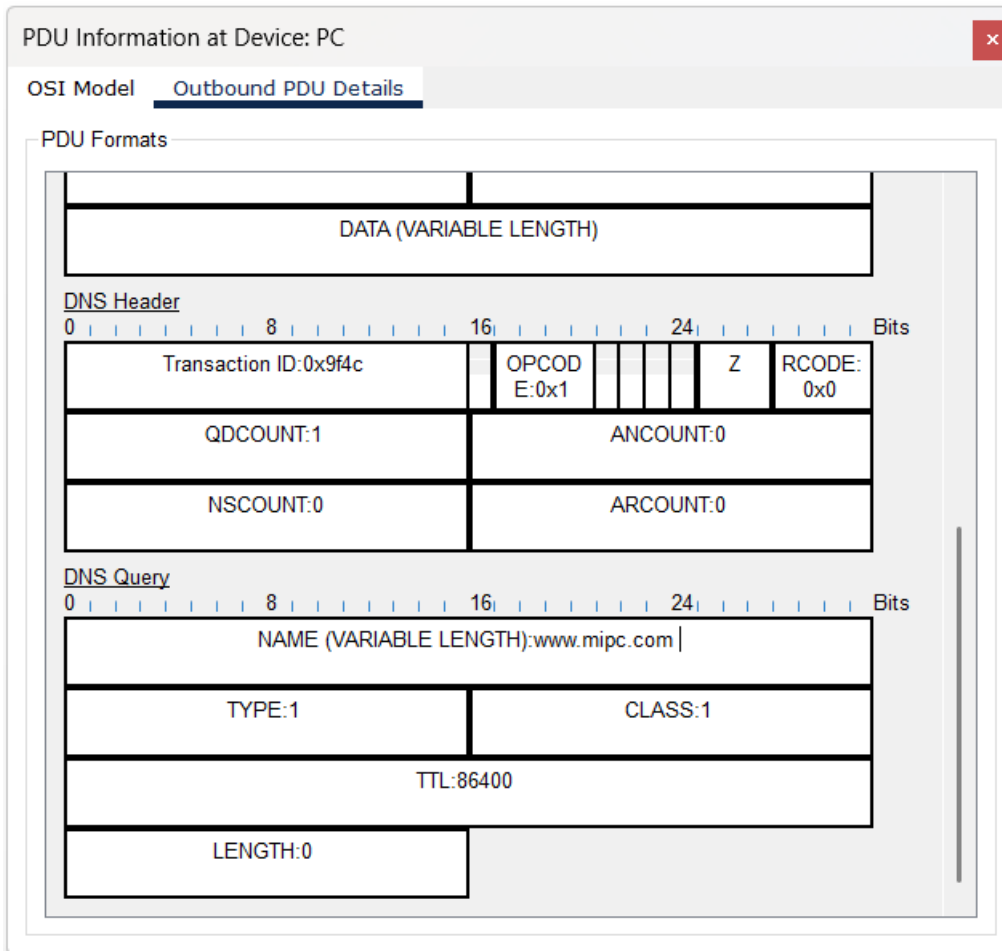
Layer 4: UDP Src Port 1025, Dst Port 53. DNS viaja sobre UDP.

2) Flujo PC–Servidor DNS: relación en cabeceras, ¿conexión?, sockets

UDP es sin conexión: no hay saludo previo ni números de secuencia; la cabecera solo tiene puertos, longitud y checksum. Es una única consulta/respuesta. Socket: `192.168.0.1:1025 (PC) ↔ 192.168.0.3:53 (DNS)`.

3) El campo DATA de la cabecera de transporte

El DATA del segmento UDP transporta el mensaje de aplicación: la consulta DNS (QDCOUNT=1, NAME=www.mipc.com, TYPE=1 registro A). Es decir, "¿cuál es la IP de www.mipc.com?". Aún ANCOUNT=0 porque es la ida.



Contenido del campo DATA: consulta DNS por www.mipc.com (TYPE 1 = A).

4) Mensaje de respuesta del servidor DNS a la PC

En la aplicación trae la respuesta: www.mipc.com → 192.168.0.2 (A Record, ANCOUNT=1). En la cabecera de transporte se invierten los puertos: origen 53, destino el puerto efímero de la PC (1025). Sigue siendo UDP, sin estado de conexión.

5) Segundo protocolo de aplicación y su transporte

El segundo es HTTP (con la IP ya resuelta, la PC pide la página). Su transporte es TCP (puerto 80).

PDU Information at Device: Servidor Web

At Device: Servidor Web
Source: PC
Destination: HTTP CLIENT

In Layers	Out Layers
Layer 7: HTTP	Layer 7: HTTP
Layer 6	Layer 6
Layer 5	Layer 5
Layer 4: TCP Src Port: 1025, Dst Port: 80	Layer 4: TCP Src Port: 80, Dst Port: 1025
Layer 3: IP Header Src. IP: 192.168.0.1, Dest. IP: 192.168.0.2	Layer 3: IP Header Src. IP: 192.168.0.2, Dest. IP: 192.168.0.1
Layer 2: Ethernet II Header 0006.2A11.C60A >> 00D0.BCA9.4EAE	Layer 2: Ethernet II Header 00D0.BCA9.4EAE >> 0006.2A11.C60A
Layer 1: Port FastEthernet0	Layer 1: Port(s): FastEthernet0

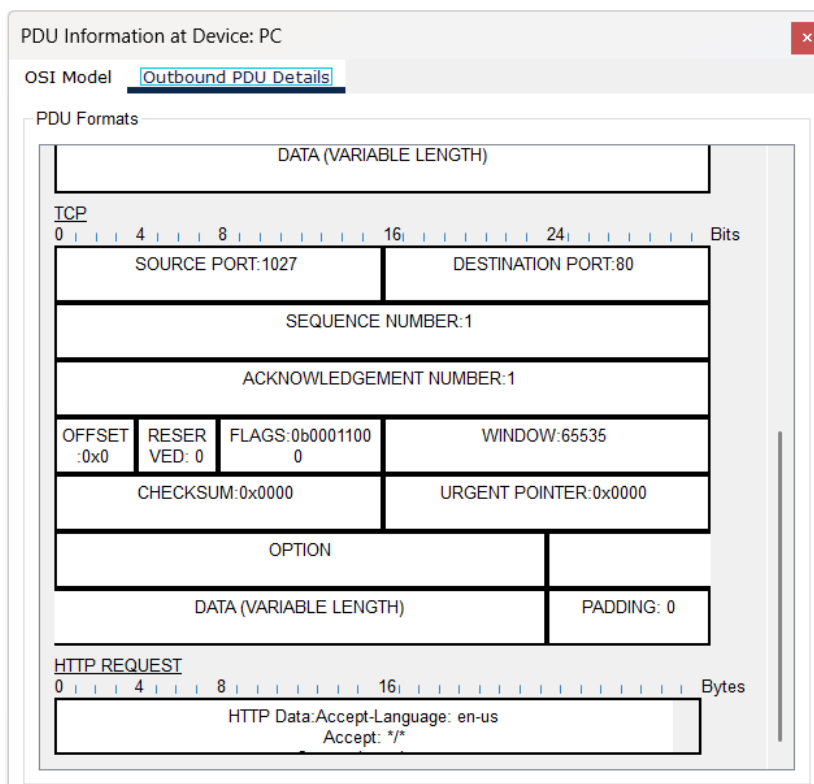
1. Sent segment information: the sequence number 1, the ACK number 102, and the data length 379.

Challenge Me << Previous Layer Next Layer >>

Layer 4: TCP puerto 80. Muestra SEQ=1, ACK=102, data length 379 (hay números de secuencia).

6) Flujo HTTP PC–Servidor Web: relación, ¿conexión?, sockets

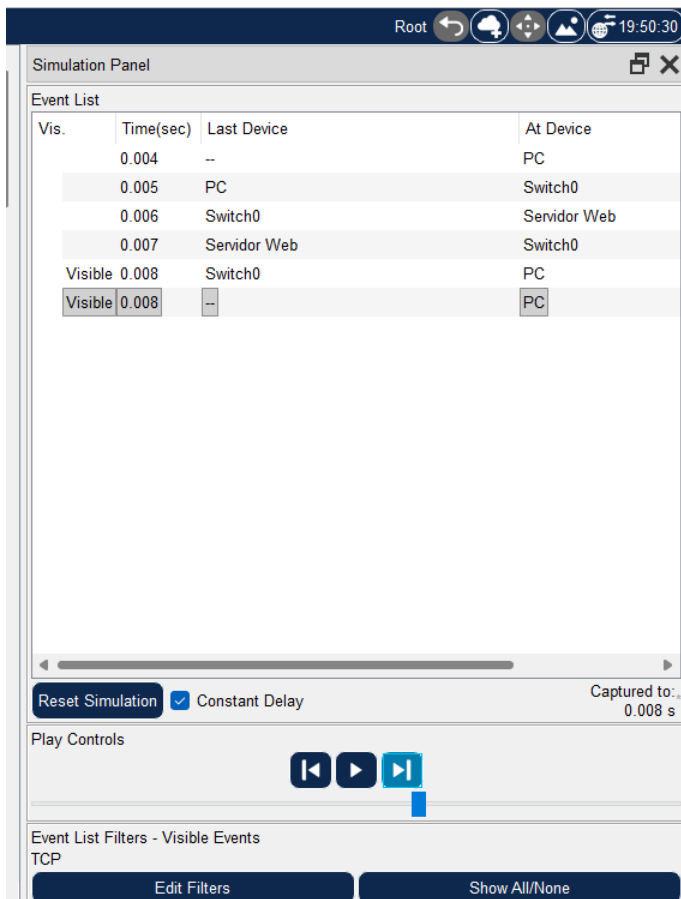
TCP es orientado a conexión: hay saludo de 3 vías. Las cabeceras se relacionan por SEQ y ACK (cada ACK = SEQ recibido + bytes). Socket: 192.168.0.1:1025 ↔ 192.168.0.2:80. Dentro del segmento viaja el pedido HTTP (GET).



Segmento TCP con FLAGS PSH+ACK que transporta el HTTP REQUEST (GET) dentro del DATA.

7) ¿Se ve el saludo de 3 vías? ¿Estamos viendo los mensajes de transporte?

No, todavía no. Con solo DNS y HTTP en el filtro vemos los mensajes de aplicación, pero no los segmentos de control de TCP (SYN, SYN-ACK, ACK). Para verlos hay que habilitar el filtro TCP.



Con el filtro TCP habilitado aparecen los segmentos de transporte del diálogo HTTP.

8) Habilitando ambos transportes (TCP y UDP): ¿aparecen nuevos mensajes?

Aparecen nuevos mensajes de TCP (SYN / SYN-ACK / ACK y el FIN del cierre). De UDP no aparecen nuevos: el que no vemos es UDP, porque es sin conexión y no genera segmentos de control aparte (el mensaje DNS ya es el segmento UDP). Socket del handshake: 192.168.0.1:1027 ↔ 192.168.0.2:80.

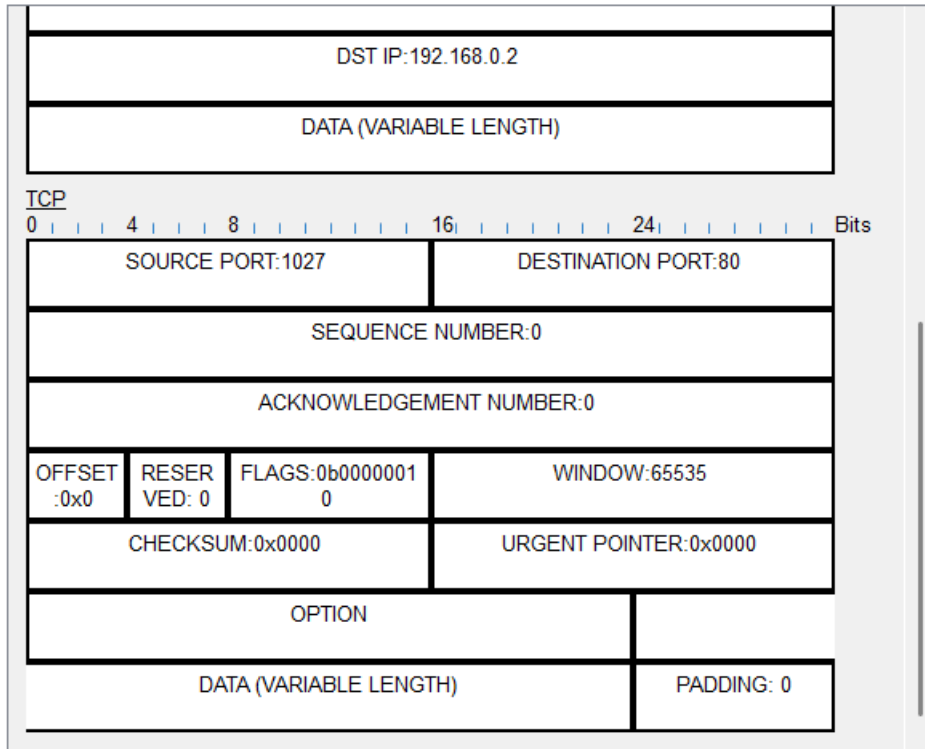
9) ISN, flags y establecimiento de la conexión

ISN de la PC = 0 (SEQ del SYN). ISN del Server = 0. Flags: PC→SRV primero SYN; SRV→PC SYN+ACK; PC→SRV ACK. La conexión queda establecida a partir de SEQ = 1 de la PC (ISN+1).

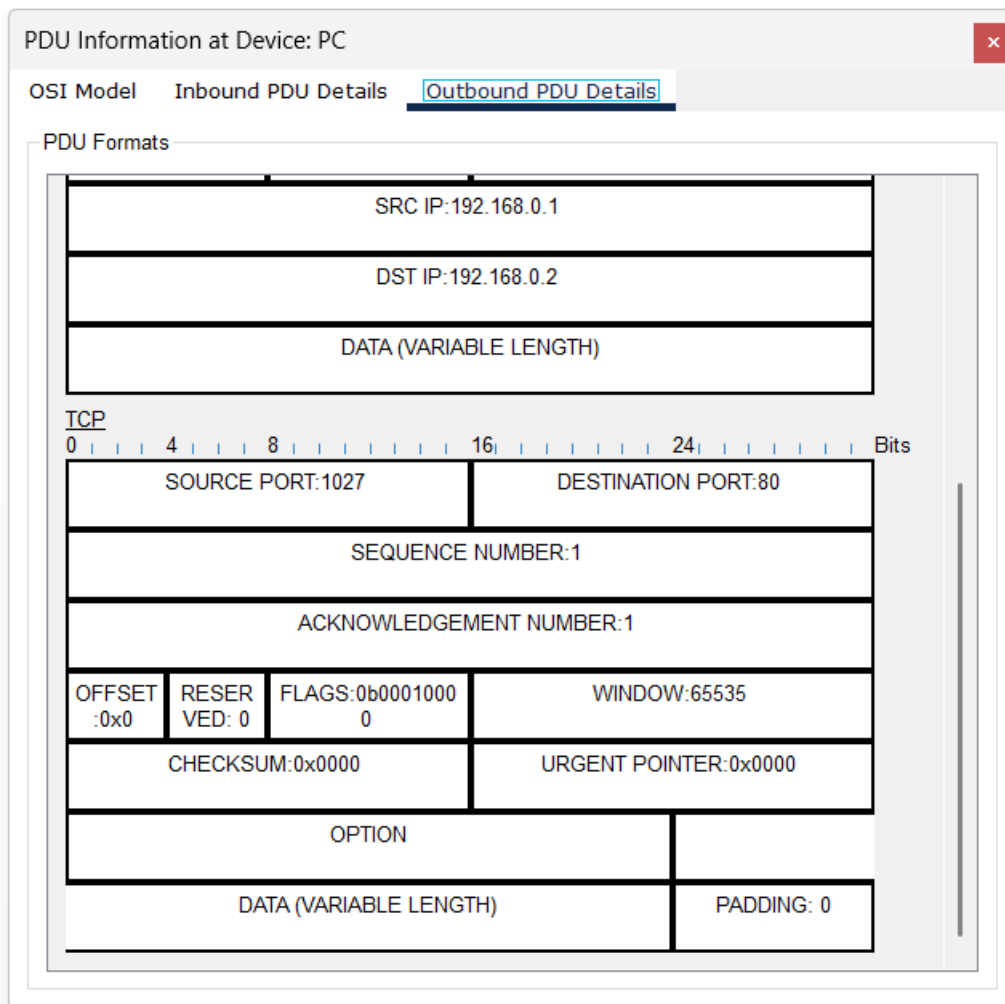
PDU Information at Device: Switch0

[OSI Model](#)
[Inbound PDU Details](#)
[Outbound PDU Details](#)

PDU Formats



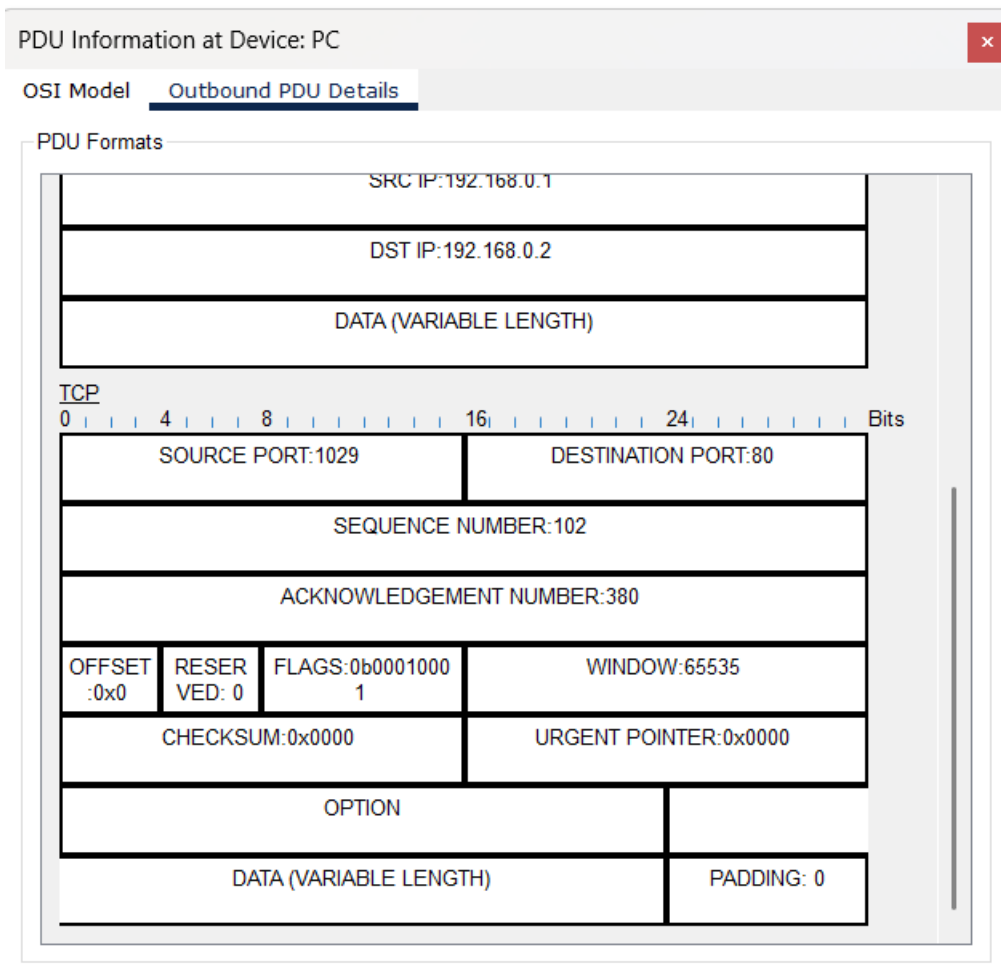
SYN: SEQUENCE NUMBER=0 (ISN de la PC), FLAGS 0b00000010 (SYN).



Segmento con SEQ=1, ACK=1, FLAGS 0b00010000 (ACK): la conexión ya está establecida.

10) Cierre de la conexión

El cierre se inicia con el FIN (FLAGS 0b00010001 = FIN+ACK) que envía la PC con SEQ = 102. La conexión se da por cerrada cuando ambos lados envían su FIN y son reconocidos con ACK.



FIN: SEQ=102, ACK=380, FLAGS 0b00010001 (FIN+ACK). Inicio del cierre.

11) netstat en el cliente y en el servidor

Muestra la conexión TCP 192.168.0.1:1029 ↔ 192.168.0.2:80. La PC en estado FIN_WAIT_1 (inició el cierre, envió FIN y espera confirmación); el servidor la muestra en CLOSED, y el puerto 443 (HTTPS) sin conexiones. DNS (UDP) no aparece porque no mantiene estado.

PC

Physical Config Desktop Programming Attributes

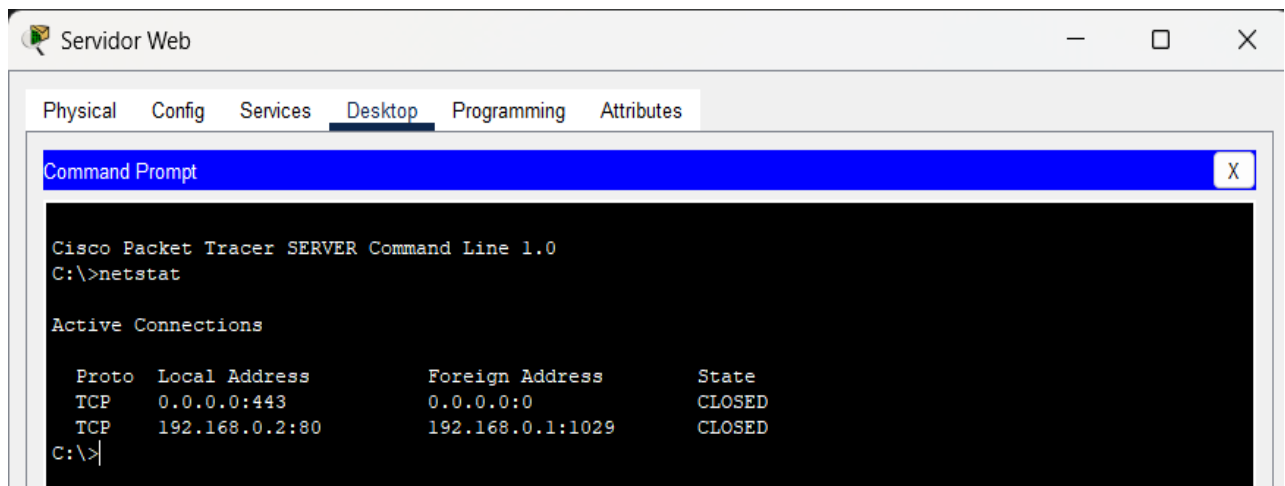
Command Prompt

```
Cisco Packet Tracer PC Command Line 1.0
C:\>netstat

Active Connections

   Proto Local Address          Foreign Address         State
   TCP    192.168.0.1:1029      192.168.0.2:80         FIN_WAIT_1
C:\>
```

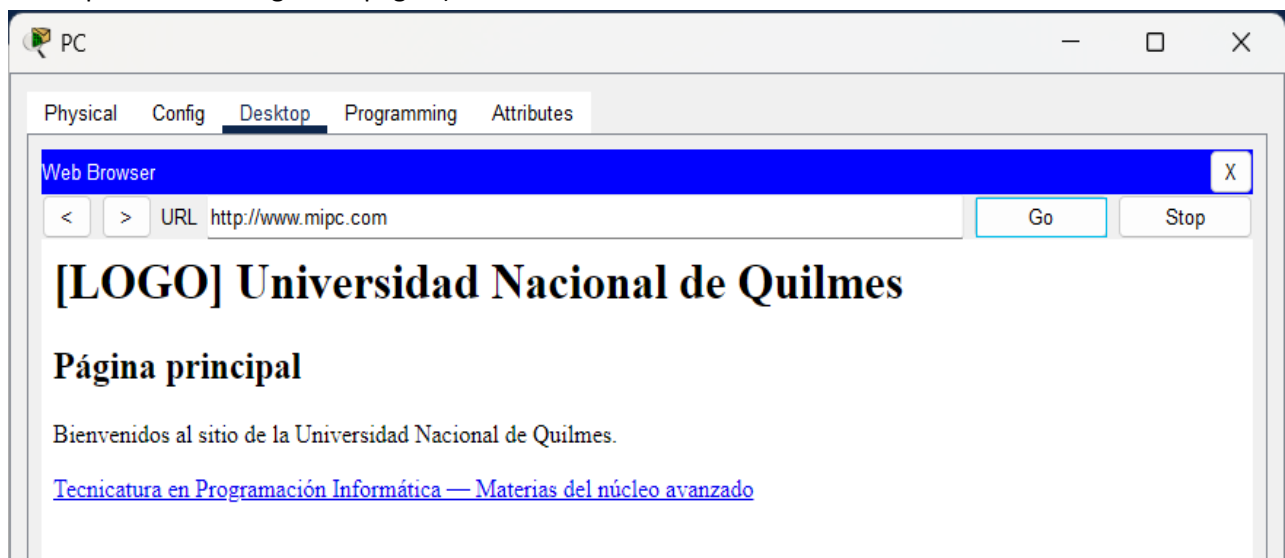
netstat del cliente (PC): conexión :1029 ↔ :80 en FIN_WAIT_1.



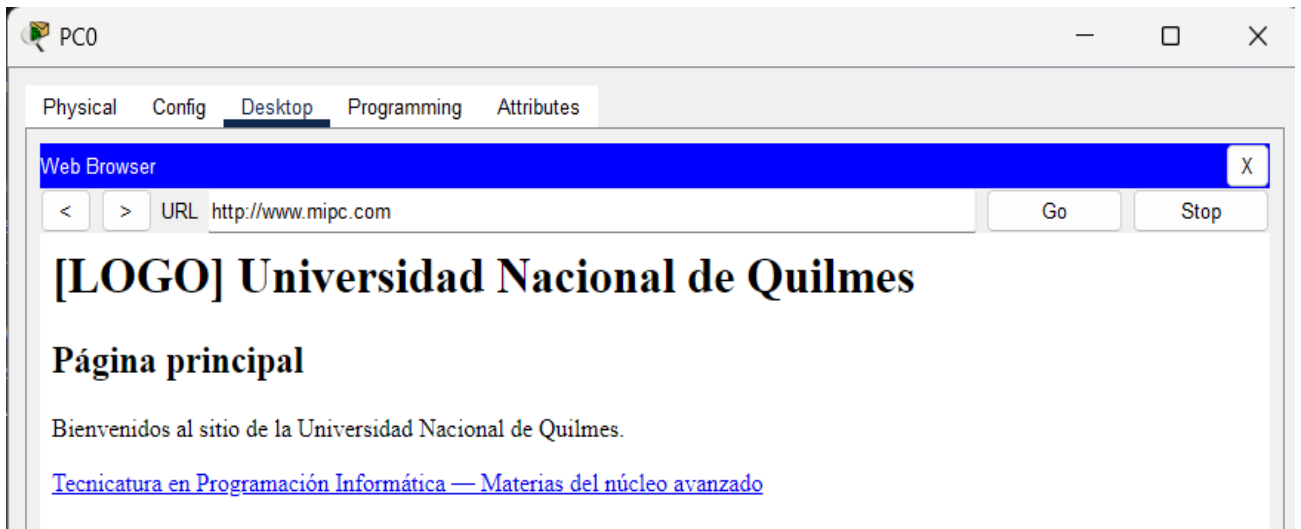
netstat del servidor: misma conexión vista desde su lado + puerto 443 (HTTPS) cerrado.

12) Segundo cliente navegando en simultáneo

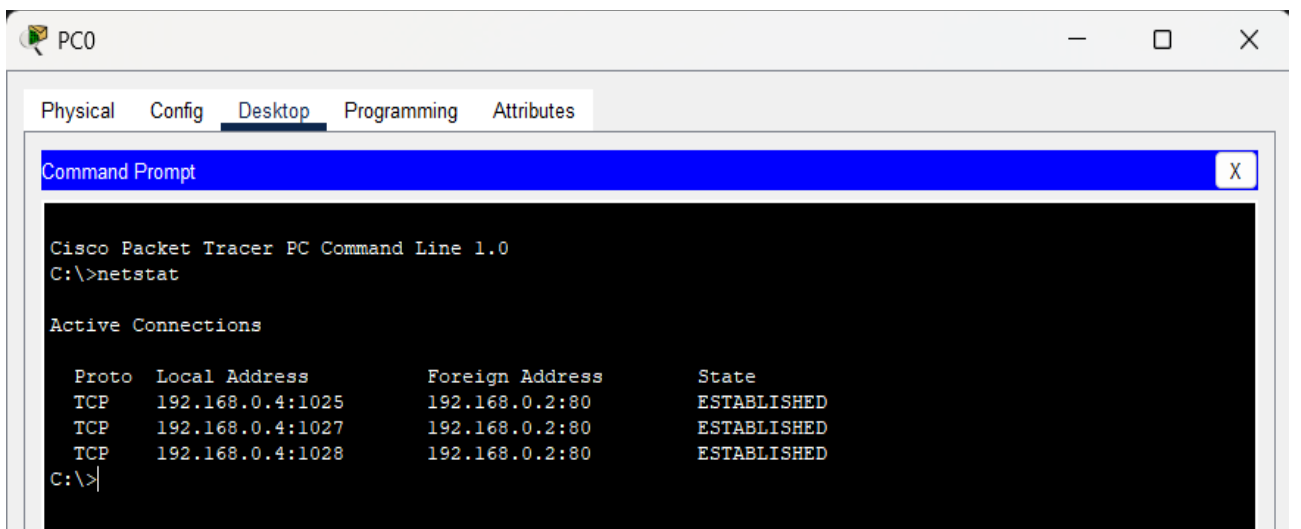
Se agrega un segundo cliente (PC0, IP 192.168.0.4, DNS 192.168.0.3). Ambos navegan a www.mipc.com. Se observan flujos independientes: cada PC hace su consulta DNS y abre su(s) conexión(es) TCP al servidor. Se establecen 2 conexiones (una por cliente; en la captura PC0 tiene varias por haber recargado la página).



PC (192.168.0.1) navegando www.mipc.com.



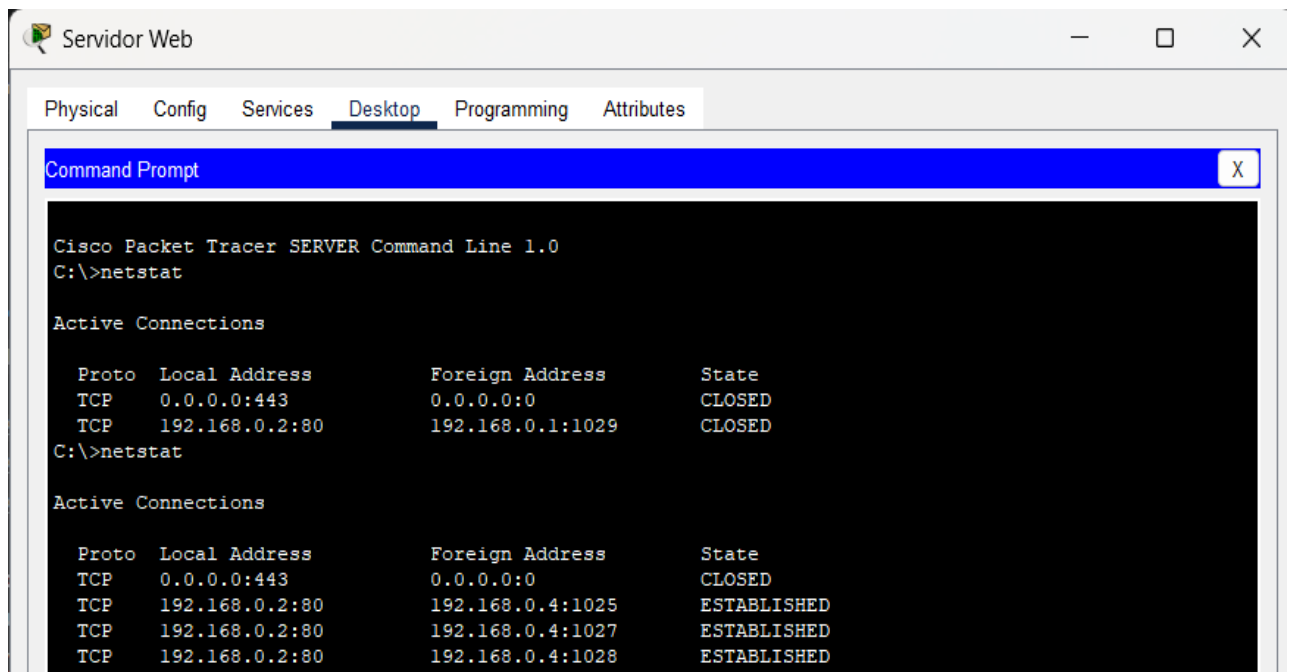
PC0 (192.168.0.4) navegando www.mipc.com.



netstat de PC0: sus conexiones al servidor (cada cliente ve solo las propias).

13) netstat con dos clientes: diferencias y conexiones existentes

El servidor muestra conexiones de los dos clientes (192.168.0.1 y 192.168.0.4) todas por el puerto 80. Cada conexión se distingue por la cupla de 4 (IP:puerto origen + IP:puerto destino), no por el puerto del servidor. Cada cliente, en su propio netstat, ve solo su conexión.



netstat del servidor: conexiones desde 192.168.0.1 y 192.168.0.4, todas al puerto 80.

Conclusión

DNS usa UDP (sin conexión, una consulta/respuesta rápida) y HTTP usa TCP (con conexión: saludo de 3 vías, transferencia con SEQ/ACK y cierre con FIN). Un mismo servidor atiende múltiples clientes por el mismo puerto 80 porque cada conexión se identifica por su cupla de sockets.